ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

Internal Consistency Is Not Evidence: Custody, Durability, and the Limits of a Tamper-Evident Log

— A log that verifies against itself has told you nothing.

Jim Schwoebel[1,2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

A tamper-evident log chains its records, roots them in a Merkle tree, and signs the root. Break any record and a link breaks. That is all true, and it is worth almost nothing against the party running the log, because that party does not break links—they recompute them. The rewritten log is not a damaged log. It is a well-formed log of different events, and every check it can perform on itself passes.

We say plainly what does the work instead. It is not cryptography; it is custody. A root that left the operator's hands before the rewrite is the only artefact that contradicts the rewrite, and a system with no such publication point is undefended no matter how carefully its proofs are implemented. We build the log anyway—chain, RFC 6962 tree, inclusion and consistency proofs verified exhaustively rather than sampled—and then make the tool print, on every root it emits, the sentence saying what it does not defend against.

Along the way we hit something we did not expect, and we think it is the more useful of our two findings. The requirement was fail-closed durability: never acknowledge a claim whose bytes are not on disk. The obvious way to test that is to make the store unavailable and check that everything is refused, and the obvious way to make it unavailable is to delete the file. That test passes while the system is broken. A process holding an open descriptor keeps writing after the path is unlinked, `fsync` returns success, and the log acknowledges claims backed by bytes nothing can ever read back. The fault injection injects no fault. We found it by luck—our assertion failed for the opposite reason—and we report the luck rather than dressing it up as method.

We also report that the schema we were implementing chains over two fields of a record rather than the record, which leaves the step index, the target resource, the policy bundle and every identity field outside the chain entirely.

Finally we derive — from the function that assigns records to trees, not from any timing — what the tree arrangement costs in linkability. A single global tree's effective anonymity set grows with the fleet; a tree per agent pins it at one agent's actions no matter how large the fleet gets, so the gap widens without bound, on any hardware. We also record that our predeclared throughput prediction failed by sevenfold at high batch size, and that the other half of the experiment — what the arrangement costs in throughput — was never actually tested, for a reason we explain rather than paper over.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript. Co-authors will be added only upon explicit consent and substantive contribution; the author list is deliberately short at this stage rather than inherited from a sibling paper.

1 The Problem

Somebody hands you a log. It records decisions an automated agent made: what it wanted to do, what the policy engine said, whether the action went through. Every record contains a hash of the record before it, so the records form a chain. The whole set is summarised by a Merkle root, and that root is signed. You check it. Every link holds. The signature is good.

What have you learned?

The honest answer is: that this log is well-formed. Not that it is true. Those are different, and the difference is the entire subject of this paper.

Here is why. Suppose the operator of the log wants record three to say something else. They are not going to alter record three and leave a broken link for you to find—that would be a strange kind of adversary, one who tampers but stops halfway. They alter record three, recompute the chain link for record four, and record five, and so on to the end. Then they sign the new root. Now the log has no broken link, because they fixed the links. It has no gap in the sequence, because they did not remove anything. The signature over the new root verifies, because they hold the key.

You are looking at a log that is, in every mechanical sense, perfect. And it is a log of events that did not happen.

I want to be careful about what is surprising here, because on one reading it is not surprising at all. Of course somebody with the signing key can sign things. That is what a key is. Everybody nods.

The trouble is that the nodding does not survive contact with a deployment. In practice you get a document that says the log is tamper-evident, and an auditor who verifies the log and reports that it checks out, and neither the document nor the auditor makes the distinction I just made. The verification step is real work, it produces a green tick, and the green tick means "well-formed," and everyone reads it as "true." The gap is not in anybody's understanding of cryptography. It is in what the tooling reports and what the report is taken to mean.

So we built the tool, and then we made the tool refuse to produce that green tick without a sentence next to it.

1.1 The one thing that does contradict a rewrite

There is exactly one artefact that catches this, and it is not clever.

If, before the rewrite, the operator published the root—handed it to a counterparty, wrote it into a transparency log, printed it and put it in a drawer—then that old root is a commitment they can no longer edit. A Merkle consistency proof between the old root and the new one either demonstrates that the new tree extends the old, or it does not exist, and against a rewritten history it does not exist. RFC 6962 defines exactly this proof for exactly this reason [4].

Notice what that costs. Not computation: the proof is logarithmic and the verification is a few dozen hashes. What it costs is a relationship. Somebody other than the operator has to have been holding something, before the fact, and be willing to say so afterwards. That is a custody arrangement and an organisational commitment, and no amount of engineering inside the log produces it.

This is the paper's central claim, and I want to state it in its blunt form before we dress it up:

> The security of a tamper-evident log against the party operating it is bounded above by the custody of its published roots, and is independent of the quality of its cryptography.

Independent. Not "mostly determined by." A log with a flawless RFC 6962 implementation and no publication point, and a log with a mediocre one and no publication point, are equally defenceless

against their operator. The cryptography is doing something—it defends against everyone who is not the operator, which is most attackers—but against the operator it contributes zero, and adding more of it continues to contribute zero.

1.2 Why this matters more for agents than it used to

Transparency logs have lived with this for a decade and solved it socially: Certificate Transparency has multiple independent logs, monitors that watch them, and auditors that gossip roots between clients [4, 5]. The design assumes the log operator is not trusted, and builds the ecosystem that makes the assumption safe.

The systems we are now being asked to log are not like that. An enterprise deploying an autonomous agent wants a record of what the agent did, for its own governance and for its regulator. The natural deployment is one log, run by the enterprise, holding records about the enterprise's own agents. Every party in that picture is the same party. The evidence that an agent did not exceed its authority is held, signed, and—if it comes to it—rewritten by the organisation whose interest is served by the answer.

I do not think this is usually malice. It is more often that nobody drew the diagram and noticed that the auditor and the audited are the same box.

2 What Others Have Built

The machinery here is old and good, and none of it is ours.

Merkle gave us the hash tree [6], which lets you commit to a set of things with one value and later prove membership without revealing the set. Haber and Stornetta gave us the linked timestamp [2], and stated the custody problem more clearly in 1991 than most systems state it now: their scheme distributes trust across clients precisely because a single timestamping service could collude with a client to backdate. That is our problem, thirty-five years earlier, and their answer was structural rather than cryptographic.

Certificate Transparency turned this into deployed infrastructure [4, 5]. The two proofs we implement are theirs: inclusion, that a record is in a tree with a given root; and consistency, that a tree of n leaves extends a tree of $m \leq n$ leaves. Crosby and Wallach worked through what tamper-evident logging costs in practice and where the naive data structures fall over [1].

On the durability side, the write-ahead log and the group commit are from database work of the seventies and eighties, with ARIES as the canonical write-up [7]. The idea we lean on—do not tell anybody the transaction committed until the bytes are on the disk—is the oldest idea in this paper.

What is newer, and more directly relevant to Section 4, is a line of work showing that the storage stack does not behave the way applications believe. Pillai and colleagues found that applications routinely depend on file-system behaviours that are not guaranteed, and that the resulting bugs survive testing because the tests share the application's assumptions [8]. Rebello and colleagues asked what happens when `fsync` itself fails and found that most applications handle it badly or not at all [9]. Our finding is a cousin of these: not that the storage stack lied, but that the *test* of the storage stack was a no-op for a reason nobody in the room had thought about.

Where we differ from all of this is emphasis rather than mechanism. The literature above mostly asks how to build the thing correctly. We are asking what a correctly built thing is worth in a deployment where the operator and the audited party coincide, and our answer is: less than the documentation says, and the shortfall is not fixable inside the tool.

3 Writing It Down Precisely

The symbols in this section will not make anything truer than the English already did. A reader who skips to Section 4 loses only precision. But it is worth being exact about which checks depend on what, because the whole argument is that two of them look alike and are not.

Let a log be a sequence of records $R = (r_0, \ldots, r_{n-1})$. Write $d(r)$ for the digest of a record's canonical form—RFC 8785 [10], computed by a shared implementation rather than a second one, for reasons the sibling paper works through—and define the chain by $h_0 = 0^{256}$ and $h_{i+1} = H(h_i \| d(r_i))$. Let $\mathrm{MTH}(R)$ be the RFC 6962 Merkle tree head over the leaf hashes of R.

Definition 1 (Internal consistency)**.** *R is internally consistent if each r_i declares chain head h_i, and the step indices of R are exactly $0, \ldots, n-1$.*

Definition 2 (Extension)**.** *R' extends R if R is a prefix of R'.*

Now the two propositions that matter, and the gap between them.

Proposition 1 (Internal consistency is cheap to forge)**.** *For any sequence of claims C and any modification of any claim in C, there is an internally consistent log recording the modified sequence, computable in time linear in $|C|$ by any party able to sign a root.*

The proof is the attack: recompute the chain from the modified record forward, rebuild the tree, sign. Nothing is hard. We ship it as a runnable example rather than as an argument, because a reader can then watch every internal check return true.

Proposition 2 (Consistency against a prior root is not)**.** *Let $\rho = \mathrm{MTH}(R)$ be published at size m. If R' of size $n \geq m$ does not extend R, then no consistency proof between (m, ρ) and $(n, \mathrm{MTH}(R'))$ verifies, under collision resistance of H.*

This is RFC 6962's property, not a new one [4]. We restate it only to make the asymmetry visible: Proposition 1 needs the signing key and nothing else, and Proposition 2 needs an artefact that escaped the operator at a particular time. The first is a computation. The second is a fact about the world.

> **The custody condition.** A log detects operator rewriting at size m if and only if some party outside the operator's control holds $\mathrm{MTH}(R_{\leq m})$ and will produce it on demand.

I have written that as a biconditional and I want to flag that the "only if" direction is the interesting one and the one we can actually argue: with no external holder, the operator's view of history is the only view, and any internally consistent log is consistent with it. The "if" direction assumes the external party is honest and available, which is an organisational assumption doing a lot of quiet work. We have not formalised what happens when the external holder is partly trusted, or when there are several disagreeing ones. That is where the Certificate Transparency ecosystem put a decade of effort [5], and we have not repeated it.

4 The Test That Proves Nothing

Now the part we did not see coming.

Separately from all the Merkle machinery, the log has a durability requirement, and it is the requirement the whole component exists for: never acknowledge a claim whose bytes are not durably written. If an enforcement point is waiting on this acknowledgement before letting an action proceed,

then acknowledging early means an action happens with no evidence that it happened. Fail-closed, no exceptions, no flag to turn it off.

The implementation is standard. Serialise the batch, one `write`, one `fsync`, and only then construct the acknowledgement. We made the acknowledgement type constructible in exactly one place in the codebase, after the sync returns, so that "acknowledge before durable" is not an expressible program rather than a discouraged one.

Then we wrote the test. The requirement says: make the evidence store unavailable, and observe that actions do not proceed. So: create the log, take the store away, submit some claims, assert that every one of them is refused.

How do you take the store away? You delete the file. That is the obvious move, and it is what we did—unlink the segment and put a directory in its place, so that any attempt to re-open it fails.

The test failed. Every claim was acknowledged.

Here is what is going on, and it is embarrassing in the specific way that good findings often are. On a POSIX system, a file descriptor refers to an inode, not to a path. Unlinking the path removes the directory entry. It does not touch the descriptor. Our committer had opened the segment at start-up and was still holding that descriptor, so:

```
write to unlinked fd: Ok(())
reopen at same path: Err(Os { code: 21, kind: IsADirectory })
```

The write succeeds. The `fsync` succeeds. The bytes really are on the disk, in an inode with no name, which the kernel will free the moment the last descriptor closes. The durability requirement is satisfied in the narrowest possible reading—the bytes were synced—and violated in every sense anybody cares about, because the evidence cannot be read back by anything, ever.

So the log acknowledges the claim. The enforcement point lets the action through. The action happens, and there is no record of it, and no error was raised anywhere in the system.

4.1 Why we want to make a fuss about this

The bug is not that unlinked descriptors keep working. That is specified behaviour—`unlink` removes the directory entry and defers destroying the file until the last descriptor is closed [3]—and any systems programmer asked directly will tell you. We confirmed it on macOS with the standalone probe above and have not run it elsewhere. The finding is about the test.

We wrote a fault-injection test. It injected no fault. The store was never unavailable; only its name was. And the assertion the test existed to check was therefore never exercised at all. Had our implementation been slightly different—had we, say, cached the acknowledgement path in a way that happened to refuse—the test would have passed, and we would have shipped a green fail-closed test over a durability path that had never once been made to fail.

That is the shape that worries me. A test that passes for the wrong reason is worse than a missing test, because the missing test is visible in a coverage report and the wrong one is not. And this particular wrong reason is attractive: "delete the file" is the first thing anybody tries.

4.2 How we actually found it, which was luck

I want to be accurate about the process, because the tidy version would be dishonest.

We did not find this by reasoning about POSIX. We found it because the assertion failed—and it failed in the direction that says "you acknowledged something you should have refused," which is alarming enough that we went looking at the mechanism instead of the implementation. If the test had passed, we would have ticked it off and moved on. The mechanism was only inspected because the outcome was already wrong.

So the honest summary is: a bug in our fault injection was caught by an unrelated bug in the same test, and if either had been absent we would have shipped. We wrote a standalone probe afterwards to confirm the POSIX behaviour directly rather than infer it from the failure, and that probe is the listing above.

4.3 The fix, and what it does not fix

The repair is not in the test. A committer that cannot re-open its store now refuses everything, because a descriptor that has outlived its path is exactly the condition under which acknowledging is unsafe. That is production behaviour, not a test affordance: the same reasoning applies if an operator moves a log directory under a running process.

What we have not done is establish that our durability path is correct under real crash conditions. Refusing when the store vanishes is one narrow case. The literature on crash-consistency testing [8, 11] exists because doing this properly means enumerating reorderings a real storage stack permits, and we have not done that. We tested that a torn trailing record is truncated and that a corrupted complete record is refused, by writing the damage ourselves. That is a simulation of a crash, not a crash.

5 How We Checked

The proofs are the part of this system where a subtle error is both easy to make and hard to notice, so they get the disproportionate share of the testing, and the testing is exhaustive rather than sampled.

For every tree size from 1 to 33, and for every leaf in the tree, we generate an inclusion proof and verify it. For every tree size from 1 to 33, and every prefix length from 1 to that size, we generate a consistency proof and verify it. Separately, the incremental root—maintained as a right fringe as leaves arrive—is compared against a recursive reference implementation written straight from the RFC, at every size.

The bound of 33 is chosen to get past the powers of two. RFC 6962 splits every interior node at the largest power of two strictly below the node's size, and an implementation that gets this subtly wrong is usually still correct at 2, 4, 8, 16 and 32. A test suite that checks those sizes finds nothing. Sizes 17 and 33 are where the fringe logic has to do something interesting, which is why the range ends just past one.

The discipline we set ourselves, and kept, was: if a proof test fails, fix the verifier against the RFC, never relax the test. This sounds obvious written down. It is less obvious at 2am when the exhaustive test fails at size 23 and the tempting move is to check a narrower property. We record it here because the sibling project's history contains exactly that failure: a verifier that compared record counts instead of roots, and consequently accepted a rewritten history, which is the attack in Section 1 passing a test suite.

We also assert the negative controls, which matter more than they look. It is easy to write a verifier that rejects a rewritten log; the empty verifier that rejects everything does that. So there is a test that an *honest* extension of the same prior root is accepted, and a test asserting that a rewritten log is accepted when no prior root is supplied—because that acceptance is the honest limitation, and if it ever changed silently, the tool would be claiming a defence it does not have.

6 The Chain That Does Not Cover The Record

One finding about the specification we were implementing, which is smaller than Section 4 but easier to act on.

The evidence schema defines the chain head as

$$h_{i+1} = H(h_i \parallel \texttt{canonical_snapshot_hash}_i \parallel \texttt{verdict}_i)$$

that is, over two fields of the record plus the previous head. Not over the record.

So the chain protects those two fields. Everything else in the record is outside it: the step index, the target resource, the policy bundle hash, the agent identifier, the initiating user, the timestamp. An operator—or anyone who can write to the store—can change which resource an action touched, or which user initiated it, or which policy bundle was in force, and every chain link still verifies, because none of those values is an input to any link.

That is very likely an oversight rather than a decision; a snapshot hash and a verdict are the two fields that feel like "the decision," and it is easy to write the definition around them. But the consequence is that a log which passes its own chain check has not established the integrity of most of its content.

We chain over the whole record digest instead: $h_{i+1} = H(h_i \| d(r_i))$. It is strictly stronger, and it is free—$d(r_i)$ is already computed to form the Merkle leaf, so this costs one hash we were performing anyway. We report the schema definition upstream as a finding rather than implement it. An implementation that honoured it would be conformant and weaker, and we would rather be non-conformant in the direction of covering more of the record.

7 What It Costs, and What We Got Wrong

An earlier draft of this paper said, in this position, that we had no performance numbers and were not going to guess. We have some now. They come from one machine, one run per cell, no warm-up and no repetitions: an Apple M5 Max with 18 logical cores running macOS, where `File::sync_data` is `F_FULLFSYNC` and a measured probe puts one call at 3438 us. Every throughput and latency number below is a fact about that machine on that afternoon, not a property of the design, and one half of what we set out to settle we did not settle at all. We say which half. The linkability numbers in Section 7.1 are the one exception, in the opposite direction: they were not timed and are not this machine's, for the reason given there.

The harness runs 1, 10, 100 and 1,000 simulated agents, each submitting 20 claims and blocking on its acknowledgement, against each of the three tree arrangements the crate implements: one global tree, one tree per agent, and one tree per agent with a periodic joint anchor — with a caveat about what that meant for the throughput half, stated in Section 7.4.

7.1 The linkability result, which is the one that matters

This is the finding. The two arrangements do not merely differ; they diverge, and the divergence grows with the fleet.

Before the table, what kind of result it is. These numbers were not measured. Our harness computes them from the agent count and the per-agent claim count alone—it hands the tree-assignment function a stipulated list of agents and action counts and never consults the throughput run's output—so Table 1 is a closed-form consequence of the assignment function, identical on any hardware, and identical had the log refused every one of the 20,000 claims the throughput sweep

submitted. It is a real result about the tree arrangement and it is not an empirical one. We say so rather than let it borrow the credibility of the timings around it, and we would rather a reader be able to re-derive the table with a pencil than believe it needed our machine.

	one global tree			one tree per agent		
agents	trees	eff. set	entropy	trees	eff. set	entropy
1	1	20	0.00	1	20.0	0.00
10	1	200	0.00	10	20.0	3.32
100	1	2,000	0.00	100	20.0	6.64
1000	1	20,000	0.00	1000	20.0	9.97

Table 1: Linkability over 20 actions per agent. "Eff. set" is the effective anonymity set: how many actions an observer who knows only which tree a record sits in cannot tell apart. "Entropy" is the tree-identity entropy in bits.

Read the two "eff. set" columns against each other. The global tree's grows with the fleet, one for one: at a thousand agents, an observer who has learned which tree a record is in has learned that it is one of twenty thousand actions, which is to say nothing. The per-agent forest's does not grow at all. It is 20.0 at one agent and 20.0 at a thousand, because every tree contains exactly one agent's twenty actions and membership in it names that agent. That flat 20.0 is the workload we stipulated, not a distribution we observed: every simulated agent submits exactly twenty claims. The effective set is the size-weighted mean of the per-tree action counts — each tree counted in proportion to how many actions it holds — which collapses to one agent's own action count only when every agent contributes the same number. Real fleets are not uniformly active, so a real deployment's per-agent figure would be that weighted mean rather than a constant, dragged above the plain average by whichever agents are busiest. What does not depend on the uniformity is the direction: whatever the distribution, the per-agent forest's effective set is bounded by the largest single agent's action count while the global tree's is the whole fleet's, so the gap still grows without bound. Meanwhile the entropy column climbs 0.00, 3.32, 6.64, 9.97 — which is $\log_2(\text{agents})$ to two decimals, and that is the point: every agent added to the fleet adds information the tree structure leaks, and none of it is shared with anybody.

So the anonymity gap between the two arrangements widens without bound as the deployment grows. This is the concrete form of an objection we had previously only stated: a per-agent forest is the arrangement people reach for because its trees are independent and shard by agent with no shared append point — whether that actually buys throughput is something we did not measure, for the reason given at the end of this section — and it is also the arrangement whose keys are agent identifiers. Anyone reaching for it because it is operationally convenient should see Table 1 first.

We must be exact about what this does *not* cover. The joint arrangement produces linkability figures identical to the per-agent one, and that is a tautology rather than a result: its tree-assignment function is byte-for-byte the same function, and our metric reads only which tree a record lands in, not when or under what anchor. It could not distinguish the two at any anchor interval or any run length. Whatever the joint anchor buys — a coarser cross-agent ordering — needs a metric over ordering to measure, and we have not written one. Global versus per-agent is the real comparison in Table 1; joint's column would be a copy of per-agent's and we have left it out rather than present it as a third data point.

7.2 The prediction we recorded, and how it failed

Before running anything, the design recorded what it expected: that with one `fsync` per batch, throughput would be mean batch size × (10^6/`sync_us`) claims per second. We recorded it in the output files precisely so that a contradiction would read as a contradiction. It contradicted.

agents	mean batch	predicted claims/s	actual claims/s	actual / predicted
1	1.0	290.8	274.1	**0.94**
10	7.14	2,077.5	1,734.4	**0.83**
100	90.9	26,440.5	10,929.7	**0.41**
1000	909.1	264,404	34,566.1	**0.13**

Table 2: The predeclared throughput prediction against what happened. It holds at batch size 1 and misses by roughly sevenfold at batch size 909.

At one agent the prediction is right to within six percent, which is the uninteresting case: batches of one, and the sync is the whole story. By a thousand agents it is wrong by a factor of about seven. Group commit does amortise — 274 claims per second becomes 34,566 — but throughput stops tracking the sync ceiling long before it reaches it.

Where the time goes is visible in the residual. Subtract the measured sync cost from the wall time per commit cycle and the remainder rises from about 0.21 ms at batch size 1 to about 22.9 ms at batch size 909. The commit loop is a single thread that deep-copies every claim in the batch, then canonicalizes, hashes and chain-links every record, before it issues the one write, and then wakes up to nine hundred blocked submitters afterwards. The copy is worth naming separately because we did not notice it until a review of this section: each claim is a ten-field structure with nine owned strings, and the loop clones all 909 of them, one at a time, before the first record is built. It is avoidable — nothing reads those claims again afterwards — and we have deliberately left it in place, because removing an allocation from the path these numbers were measured on would invalidate the table it appears under. Reporting it is the correct outcome here; fixing it is a change to make before the next run, not during the one being published. All of that per-record work is serial, in one thread, and grows with batch size in the way the residual does. So our account of Table 2 is that the prediction's error was not in the sync cost but in assuming everything else rounded to zero.

We call that an account rather than a finding, and here is the caveat that forces the distinction. Our `sync_us` figure comes from a probe that writes eleven bytes and syncs, fifty times over. A real batch of 909 records writes hundreds of kilobytes in one write–sync pair, and `F_FULLFSYNC`'s cost is not independent of how many dirty bytes it has to push. So the probe's figure is a lower bound on what a large batch's sync really costs, and the prediction over-predicts at large batch size partly *by construction*. Concretely: the arithmetic that says sync accounts for roughly 13% of wall time at a thousand agents is an upper bound on the non-sync residual, not a decomposition of wall time into a sync part and a CPU part. Some of the other 87% may be real sync cost the probe did not see. "Serial per-record work dominates" is our best account of the shape in Table 2; it is not an attributed percentage split, and we would rather say so than publish a pie chart we cannot defend.

7.3 Latency, and which number is the honest one

Two latency figures come out of this run and they mean very different things.

The unloaded one is the single-agent row: 3.9 ms at the median and 4.2 ms at the 99th percentile, with one claim in flight at a time. That is essentially one `F_FULLFSYNC` and a little bookkeeping,

and it is the number an architect sizing a fail-closed logging budget should use, because it is what the log costs to serve a request when nothing is queued behind it.

The other is 52.3 ms at the 99th percentile with a thousand agents, and it is a *closed-loop saturation* figure, not a service time. Every simulated agent submits with zero think time and blocks on its acknowledgement, so offered load is unbounded by construction and the system is saturated the moment more than a handful of agents are running. Little's law confirms it arithmetically: 34,566 claims per second times a median latency of 27.1 ms is about 937 requests in flight, against 1,000 agents. The tail we measured is concurrency divided by throughput — queueing delay we manufactured — and quoting it as "the log's p99" would be a misreading of our own harness. It answers "how bad does it get under unbounded load," which is a real question, but a different one.

7.4 The half we did not measure

We set out to answer two questions about the tree arrangement and answered one. The throughput half of the experiment never tested the topology axis at all.

The reason is mundane and worth stating exactly: in the throughput harness the topology is used in one place, to write a label into the output file. The commit path builds no tree. There is no `Forest` insertion, no root recomputation, nothing on the write path that a topology could make faster or slower. The three throughput sweeps in our results directory are therefore the same experiment run three times and labelled differently, and the harness contains no mechanism by which a real topology effect could have appeared.

We are not claiming that topology has little effect on throughput. We did not test it. That distinction is the whole reason this subsection exists. What the three runs are good for is a noise estimate on identical code: the spread between them is 4.3%, 30.3%, 19.7% and 2.0% at 1, 10, 100 and 1,000 agents respectively. Three back-to-back runs of the same code on the same machine can differ by thirty percent, which is the precision every figure in this section should be read at — and quoting the tightest row as "our measurement precision" would be cherry-picking.

The same fact carries a second consequence: every throughput number here excludes Merkle tree maintenance entirely. A deployment that maintains the forest incrementally alongside each commit would be slower than every figure above, by an amount we did not measure.

Finally, on portability. Do not carry these numbers to another platform without carrying the sync call with them. On macOS `File::sync_data` is `F_FULLFSYNC` and forces a drive-cache flush; on Linux the same Rust call is `fdatasync`, which on typical hardware does not. A Linux run of this harness may well look an order of magnitude faster while providing *weaker* durability, and a comparison that does not quote both the call and its measured cost is not a comparison of anything.

8 What We Still Don't Know

The single largest gap is the one this paper is about, and we have not closed it: we have no publication point. Everything here reduces to the custody condition, and the custody condition is an arrangement between organisations that a Rust crate cannot create. What the crate can do—and does—is refuse to let a root be printed without the sentence saying so, so that nobody reads a green tick as more than it is. That is a documentation defence against a structural problem, and I would not want to oversell it.

We do not know what the tree topology costs in throughput. Section 7 reports numbers for all three arrangements and then explains why they are not a comparison: the commit path we measured builds no tree, so the topology never reached the code whose speed we were timing. The question the sweep was designed to answer is still open, and answering it means building a harness

that maintains the forest on the write path. Until somebody does that, nobody — including us — should read our three throughput columns as evidence that the choice is cheap.

Nor do we know what the joint anchor buys. Section 7 derives the gap between a global tree and a per-agent forest, and that gap is real and grows with the fleet. The joint arrangement sits between them by design, and our metric cannot see it: the metric reads tree membership, and joint's membership is per-agent's membership. Measuring the coarser cross-agent ordering that the periodic anchor is supposed to provide needs a metric over ordering, which does not exist yet.

We also do not know how the throughput and latency figures behave on a second machine, or on a second run. Section 7's timings are one host, one run per cell, and its own noise estimate says three back-to-back runs of identical code can differ by thirty percent. Treat every timing there as an order of magnitude with a plausible leading digit, not as a benchmark result. The linkability figures are not subject to this caveat and are not helped by it either: they would be the same on any machine because no machine produced them. And whether the durability requirement is compatible with the action rates people want from agents is *still* not something we can answer in general — we can now say what one machine did, which is a different and much smaller claim.

The default build signs with a software key, and we should be blunt that this means the whole system as shipped has no defence against its operator at all—the custody argument above is not mitigated by the signer, it is the reason the signer's weakness does not much matter yet. A key in a trusted execution environment is no longer future work in the implementation: there is now one behind a compile-time feature, written to bind the key to a quote through the hash of its entire `SubjectPublicKeyInfo` and to refuse to construct a signer at all if the platform will not produce that quote. We are not reporting that it works. It has never run inside a confidential-computing guest; only its failure paths are exercised, offline, and whether real attestation hardware accepts the `report_data` we derive is unverified. The default build is unchanged, and cannot express hardware custody at all.

None of that touches the attack. A key in a trusted execution environment changes who can produce a signature, not who can publish a root, so it narrows the attack without closing it—and the part it fails to close is the part this paper is about. An operator holding a hardware-bound key can still alter a past record, relink the chain, and re-sign the result; what comes out is a log that verifies internally, carries a valid signature, and now also carries a hardware-custody label. The A9 rewrite of Section 1 survives the hardware unchanged, and a reader who takes the presence of a signer that asks for hardware attestation for a defence against the operator has read it exactly backwards.

Finally, on the finding in Section 4: we do not know how common it is. We found one instance, in our own code, by accident. The reason we think it generalises is that the reasoning that produced our broken test—"to make a resource unavailable, delete it"—is not idiosyncratic, and the failure is silent. But we have not surveyed other durability test suites for the same pattern, and until somebody does, "we think this is common" is a hypothesis and should be read as one.

Availability

The implementation, the runnable demonstration of the rewrite attack, and the source of this paper are at `https://github.com/Task-force-for-AI-agents-in-Healthcare/ephemeris`, under Apache-2.0.

References

[1] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009.

[2] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.

[3] IEEE and The Open Group. *IEEE Std 1003.1-2017, Standard for Information Technology — Portable Operating System Interface (POSIX)*, 2018.

[4] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, 2013.

[5] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, IETF, 2021.

[6] Ralph C. Merkle. Protocols for public key cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy*, pages 122–134, 1980.

[7] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[8] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 433–448, 2014.

[9] Anthony Rebello, Yuvraj Patel, Ramnatthan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Can applications recover from fsync failures? In *Proceedings of the 2020 USENIX Annual Technical Conference*, 2020.

[10] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.

[11] Mai Zheng, Joseph Tucek, Dachuan Huang, Feng Qin, Mark Lillibridge, Elizabeth S. Yang, Bill W. Zhao, and Shashank Singh. Torturing databases for fun and profit. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 449–464, 2014.